

Software Verification and Validation

Part Time Lecturer
Talimul Bari Shreshtho

Islamic University of Technology

May 16, 2026



Table of Contents

1 Introduction to Alloy

Modeling Primitive Relations

- Model parent-child relationships as primitive relations
- Model spousal relationships as primitive relations

Derived Relations

- Define relationships such as siblings using existing relations

Key Idea

Complex relationships can be derived from simpler primitive relations.

Constraints in Alloy Models

Biological Constraints

- Expressed using first-order logic predicates
- Example:
 - A person has only one mother

Social Constraints

- Additional rules about relationships
- Example:
 - A wife cannot also be a sibling

Goal

Use logical constraints to eliminate invalid system states.

Checking Derived Properties

- Verify whether certain relationships can exist

Example Property

No person has a wife with whom he shares a parent

Using Alloy Analyzer

- Automatically searches for counterexamples
- Finds violations of constraints
- Helps validate system models

Key Insight

Alloy is not just for modeling — it is also a powerful verification tool.

Main Components of an Alloy Model

Core Idea

- In Alloy, everything is built from:
 - Atoms
 - Relations

Atoms

- A primitive entity in the model
- **Indivisible**: cannot be broken into smaller parts
- **Immutable**: does not change over time
- **Uninterpreted**: has no built-in meaning (unlike numbers)

Definition

- A relation is a structure that connects atoms
- It is a set of tuples
- Each tuple is a sequence of atoms

Example

- Parent = set of pairs (Person, Person)
- Each pair represents a relationship between two atoms

Key Insight

All Alloy models are ultimately built using atoms and relations.

Main Components of an Alloy Model

Core Building Blocks

- Signatures
- Fields
- Predicates
- Functions
- Facts
- Assertions
- Commands and Scopes

Key Idea

Alloy models are constructed using these components to define the structure, behaviour, and constraints of a system.

Overview of Alloy Components

Structural Elements

- **Signatures:** Define basic types (sets of atoms)
- **Fields:** Define relations inside signatures

Behavioral Elements

- **Predicates:** Define reusable constraints
- **Functions:** Return computed values

Constraint Elements

- **Facts:** Always-true constraints in the model
- **Assertions:** Properties to be checked

Execution Control

- **Commands and Scopes:** Control analysis and search space

Basic Idea

- A **signature** introduces a set of atoms
- Example:

`sig A {}`

Extension of Signatures

- A signature can be defined as a subset of another signature

`sig A1 extends A {}`

- Means: $A1 \subseteq A$

Signature Hierarchies (Example 1)

Code

```
sig A {}  
sig B {}  
sig A1 extends A {}  
sig A2 extends A {}
```

Meaning

- $A1 \subseteq A$
- $A2 \subseteq A$
- $A1$ and $A2$ are disjoint subsets of A
- B is independent of A

Abstract Signatures and Subsets

Code

```
abstract sig A {}  
sig B {}  
sig A1 extends A {}  
sig A2 extends A {}  
sig A3 in A {}
```

Meaning

- **abstract sig A**: A cannot have atoms directly, only children
- $A1, A2 \subseteq A$
- $A3 \subseteq A$ (subset, not necessarily partitioned)
- B is independent of A

Key Insight

extends creates disjoint subtypes, while **in** defines a general subset.

Basic Idea

- Relations are declared as **fields** of signatures

$$\text{sig } A \{ f: e \}$$

- Introduces a relation $f \subseteq A \times e$
- e is an expression representing a type or product of types

Binary Relation

sig A { f1: B }

- $f1 \subseteq A \times B$

Ternary Relation

sig A { f2: B $\rightarrow$ C }

- $f2 \subseteq A \times B \times C$

Fields: Example Model

```
abstract sig Person {  
    Children: Person,  
    Siblings: Person  
}
```

```
sig Man, Woman extends Person {}
```

```
sig Married in Person {  
    spouse: Married  
}
```

Interpretation

- Person has relations: Children and Siblings
- Man and Woman are disjoint subtypes of Person
- Married is a subset of Person with a self-relation (spouse)

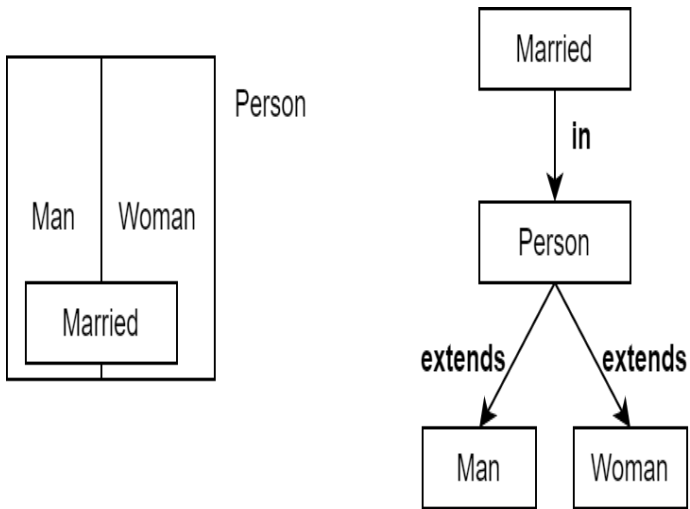


Figure 1: Family Structure

Multiplicity specifies how many values a relation can have

Four Multiplicities

- **set**: any number (0 or more)
- **some**: one or more
- **lone**: zero or one
- **one**: exactly one

Default Multiplicity

- The default is **one**

Multiplicity in Signatures and Fields

1. Multiplicity on Signatures

`m sig A {}`

- Controls how many atoms can exist in A

2. Multiplicity on Fields

`sig A { f: m e }`

`sig A { f: e1 m -> n e2 }`

- Restricts number of related atoms
- Applies on both ends of a relation

Key Idea

Multiplicity controls cardinality constraints in Alloy models.

Important Note

- Multiplicities can also be applied to full relational expressions
- This helps constrain derived relations as well

Example Use

Used to enforce constraints like:

- Each person has exactly one mother
- A node may have at most one parent

Alloy Model

```
abstract sig Person {  
    Children: set Person,  
    Siblings: set Person  
}  
  
sig Man, Woman extends Person {}  
  
sig Married in Person {  
    spouse: one Married  
}
```

Person Signature

- Each Person can have:
 - **Children:** any number (including zero)
 - **Siblings:** any number (including zero)

Subtypes

- Man and Woman are subsets of Person
- They inherit Children and Siblings relations

Marriage Constraint

- Each Married person has exactly **one spouse**
- Enforced using **one**

Multiplicity in Modeling

- **set** allows flexibility (many or none)
- **one** enforces strict pairing (exactly one)
- Helps encode real-world constraints precisely

Important Note

- Alloy models are not just structures
- They are **constrained mathematical systems**

Common Logical Connectives

- **not** (!): Negation
- **and** (&&): Conjunction
- **or** (||): Disjunction
- **implies** ($\Rightarrow$): Implication
- **else**: Alternative choice
- $\Leftrightarrow$: Equivalence

Key Idea

These operators are used to build constraints in Alloy specifications.

Quantifiers express properties over sets

- They allow reasoning about all, some, or no elements

Universal Quantifier

$\text{all } x: S \mid F$

- F holds for every $x \in S$

Existential Quantifier

$\text{some } x: S \mid F,$

- F holds for at least one $x \in S$

Other Quantifiers in Alloy

No Elements

$\text{no } x: S \mid F$

- F holds for no $x \in S$

At Most One

$\text{lone } x: S \mid F$

- F holds for at most one $x \in S$

Exactly One

$\text{one } x: S \mid F$

- F holds for exactly one $x \in S$

Set Operators in Alloy

Everything is a Set in Alloy

Set Operations

- $\cup$ (union): combines elements
- $\&$ (intersection): common elements
- $-$ (difference): elements in one set but not the other
- `in`: subset relation
- $=$: equality
- $\neq$: disequality

Example

`Married Men = Married & Man`

Key Insight

All Alloy expressions ultimately evaluate to sets.

Relational Operators

- $\rightarrow$: arrow (cross product)
- $\sim$: transpose (reverse relation)
- $.$: dot join (relational join)
- $[]$: box join
- $\wedge$: transitive closure
- $*$: reflexive-transitive closure
- $<:$: domain restriction
- $:>$: image restriction
- $++$: override

Key Insight

These operators allow compact expression of complex structural constraints in Alloy models.

Arrow Operator (Cross Product)

Idea

- The arrow operator $\rightarrow$ builds a cross product of relations
- It combines every tuple from the left with every tuple from the right

Definition

$$p \rightarrow q$$

- p and q are relations
- Result is all possible concatenations of tuples from p and q

Example of Arrow Operator

Given

$$\text{Name} = \{(N0), (N1)\} \quad \text{Addr} = \{(D0), (D1)\} \quad \text{Book} = \{(B0)\}$$

Compute

$$\text{Name} \rightarrow \text{Addr} = \{(N0, D0), (N0, D1), (N1, D0), (N1, D1)\}$$

$$\text{Book} \rightarrow \text{Name} \rightarrow \text{Addr}$$

$$= \{(B0, N0, D0), (B0, N0, D1), (B0, N1, D0), (B0, N1, D1)\}$$

Key Insight

The arrow operator constructs all possible combinations across relations.

Transpose Operator ($\sim$)

Idea

- The transpose operator reverses a relation
- It takes the mirror image of each tuple

Definition

$$\sim p$$

- Reverse the order of atoms in each tuple of p

Example

$$p = \{(a0, a1, a2, a3), (b0, b1, b2, b3)\}$$

$$\sim p = \{(a3, a2, a1, a0), (b3, b2, b1, b0)\}$$

Meaning of $\sim$ children

Interpretation

- If `children` relates a parent to their children:

$$\text{children} \subseteq \text{Person} \times \text{Person}$$

Then:

$$\sim \text{children}$$

- Reverses the relationship
- Maps each child to their parent(s)

Meaning

- `children`: $\text{parent} \rightarrow \text{child}$
- `$\sim$ children`: $\text{child} \rightarrow \text{parent}$

Key Insight

A transposition simply reverses the direction of a relationship.

Dot Join Operator

$$p.q$$

- p and q are relations
- At least one of them must be non-unary
- Join combines tuples when their connecting atoms match

Intuition

- Follow one relation by another
- Similar to chaining relationships together

Join Mechanism

Given two tuples:

$$(a_1, a_2, \dots, a_m) \quad (b_1, b_2, \dots, b_n)$$

Case 1: No Match

If: $a_m \neq b_1$, then the join is undefined.

Case 2: Match Exists

If: $a_m = b_1$, then the join becomes:

$$(a_1, a_2, \dots, a_{m-1}, b_2, \dots, b_n)$$

Key Idea

The last atom of the first tuple must match the first atom of the second tuple.

Example of Relational Join

Given:

$$p = \{(John, Mary), (Sam, Alex)\} \quad q = \{(Mary, Bob), (Alex, Eve)\}$$

Compute

$$p \cdot q$$

Result

$$p \cdot q = \{(John, Bob), (Sam, Eve)\}$$

Explanation

- $(John, Mary)$ joins with $(Mary, Bob)$
- $(Sam, Alex)$ joins with $(Alex, Eve)$

to maps a message to the name it should be sent to

address maps names to addresses

to = $\{(M0, N0), (M0, N2), (M1, N2), (M2, N3)\}$

address = $\{(N0, D0), (N0, D1), (N1, D1), (N2, D3)\}$

to.address maps a message to the addresses it should be sent to

to.address = $\{(M0, D0), (M0, D1), (M0, D3), (M1, D3)\}$

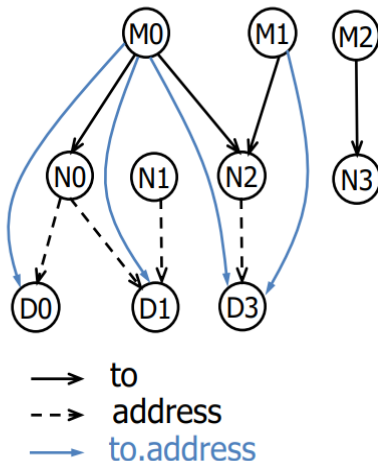


Figure 2: Relational Composition

Using Join in Family Structures

Finding Matt's Children

```
Matt.children
```

- Follow the `children` relation once

Finding Matt's Grandchildren

```
Matt.children.children
```

- Follow the `children` relation twice

Key Insight

Dot join allows navigation through chains of relationships.

Constraint: Marriage Rules

Every married man has a wife and every married woman has a husband

```
all p: Married |  
    (p in Man => p.spouse in Woman)  
    and  
    (p in Woman => p.spouse in Man)
```

Meaning

- Married men can only have wives
- Married women can only have husbands

Constraint: Spouse Cannot Be a Sibling

no p : Married | $p.spouse$ in $p.siblings$

Meaning

- No married person can have a spouse who is also their sibling

Logical Interpretation

$$\forall p \in Married, p.spouse \notin p.siblings$$

Key Insight

Alloy constraints eliminate invalid system states automatically.

Box Join Operator

$$p[q]$$

Meaning

- Semantically identical to dot join
- Arguments appear in reverse order

$$p[q] \equiv q.p$$

Key Idea

Box join is simply an alternative notation for relational join.

Examples of Box Join

Matt's Children

```
children[Matt]
```

Equivalent to:

```
Matt.children
```

Matt's Grandchildren

```
children[children[Matt]]
```

Equivalent to:

```
Matt.children.children
```

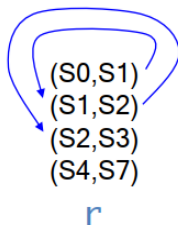
Key Insight

Repeated joins allow navigation through chains of relationships.

Transitive Closure

$\wedge r$

- Intuitively, the transitive closure of a relation $r : S \times S$ is what you get when you keep navigating through r until you can't go any farther



$(S0, S1)$

$(S1, S2)$

$(S2, S3)$

$(S4, S7)$

$(S0, S2)$

$(S0, S3)$

$(S1, S3)$

$\wedge r$

$$\wedge r = r + r.r + r.r.r + \dots$$

Using Transitive Closure

Matt's Descendants

$Matt. \sim children$

- All children, grandchildren, great-grandchildren, ...

Matt's Ancestors

$Matt.^{\sim}(children)$

- All parents, grandparents, great-grandparents, ...

Key Idea

Transitive closure follows a relation repeatedly until no further elements can be reached.

Constraint: No Cyclic Ancestry

no p : Person | p in $p.^{\sim}(\sim\text{children})$

Meaning

- No person can be their own ancestor

Logical Interpretation

$$\forall p \in \text{Person}, p \notin p.^{\sim}(\sim\text{children})$$

Key Insight

This prevents cycles in the family ancestry graph.

Reflexive Transitive Closure

$$*r = \wedge r + \text{iden}$$

(S0,S1)
(S1,S2)
(S2,S3)
(S4,S7)
 r

(S0,S1)
(S1,S2)
(S2,S3)
(S4,S7)
(S0,S2)
(S0,S3)
(S1,S3)
(S0,S0)
(S1,S1)
(S2,S2)
(S3,S3)
(S4,S4)
(S7,S7)
 $\wedge r$
 $*r$
 iden

Domain and Image Restrictions

Purpose

- Restriction operators filter relations
- They select tuples based on domain or the image

Domain Restriction

$$s <: r$$

- Keeps tuples of r starting with elements in s

Image Restriction

$$r :> s$$

- Keeps tuples of r ending with elements in s

Key Idea

Restrictions help extract meaningful subsets of relations.

Examples of Restrictions

Given

$$\text{Man} = \{(M0), (M1), (M2), (M3)\} \quad \text{Woman} = \{(W0), (W1)\}$$

$$\text{children} = \{(M0, M1), (M0, M2), (M3, W0), (W1, M1)\}$$

Domain Restriction

$$\text{Man} <: \text{children} \quad // \text{ father-child relation}$$

$$= \{(M0, M1), (M0, M2), (M3, W0)\}$$

Image Restriction

$$\text{children} :> \text{Man} \quad // \text{ parent-son relation}$$

$$= \{(M0, M1), (M0, M2), (W1, M1)\}$$

Override Operator ($++$): $p ++ q$

Idea

- Combines two relations like union
- But **q overrides p** when keys match

Definition

$$p ++ q = p - (\text{domain}(q) <: p) + q$$

- Remove tuples in p that conflict with q
- Then add all tuples from q

Override Operator Example

Given

$\text{oAddr} = \{(N0, D0), (N1, D1), (N1, D2)\}, \text{nAddr} = \{(N1, D4), (N3, D3)\}$

Compute

$\text{oAddr} ++ \text{nAddr}$

Step intuition

- Remove entries from oAddr with same first element as nAddr
- Replace them with new values

Result

$\{(N0, D0), (N1, D4), (N3, D3)\}$

Operator Precedence in Alloy

From highest to lowest precedence

- `!` (negation)
- `~` (transpose)
- `*`, `^` (reflexive-transitive closure, transitive closure)
- `.` (dot join)
- `[]` (box join)
- `<:` (domain restriction)
- `:>` (image restriction)
- `->` (cross product)
- `+` `-` (union, difference)
- `&` (intersection)
- `++` (override)
- `=` `!=` `in` (relational comparison)
- `(and)`
- `||` (or)
- `=>` `<=>` (implication, equivalence)

Constraints in Family Model

1. A person has at most one father and one mother

```
all p: Person |  
  (lone (children.p & Man)) and  
  (lone (children.p & Woman))
```

Alternative formulation using restriction

```
all p: Person |  
  (lone (Man <: children).p) and  
  (lone (Woman <: children).p)
```

Sibling Constraint

Definition of siblings

- Two people are siblings if they share the same parents
- A person is not their own sibling

Alloy constraint

```
all p: Person |  
    p.siblings =  
        {q: Person | p.parents = q.parents} - p
```

Key Idea

Siblings are derived relations, not primitive ones.

Facts in Alloy

Idea

- Facts are constraints that always hold in a model

Syntax

```
fact Name {  
    F1  
    F2  
}
```

Meaning

- Alloy only generates instances that satisfy all facts
- Facts define global system rules

Purpose

- Used to generate example instances of a model

Syntax

```
run { Predicate } for Scope
```

Meaning

- Finds a model that satisfies the given predicate + facts
- Used for exploration and testing

Assertions in Alloy

Purpose

- Assertions express properties that should always hold

Syntax

```
assert Name {  
    Property  
}
```

```
check Name for Scope
```

Meaning

- Used to verify correctness of a model
- Alloy tries to find counterexamples

Functions in Alloy

Purpose

- Functions define reusable expressions

Syntax

```
fun name[x: S]: T {  
    expression  
}
```

```
fun sisters [p: Person] : Woman {  
    {w: Woman | w in p.siblings}  
}
```

Meaning

- Computes a value from inputs
- Cannot contain constraints, only expressions

Purpose

- Predicates define reusable constraints

Syntax

```
pred name[x: S] {  
    constraints  
}
```

Meaning

- Can be used inside run or fact blocks
- Does not return values, only true/false conditions

Thank You

Thank you for your attention!

`talimulbari@iut-dhaka.edu`